ERUDITE ANGULAR WEB APPLICATION

CHAPTER 1

Installation, Workspace & First Angular Component

Layman explanations • Analogies • 3 interactive scenarios • Source ZIP



TYPESCRIPT
Data + actions

HTML
Page structure

SCSS
Visual appearance

ANGULAR
Connects the parts

44 visual pages | Module/chapter folder system | Full Chapter 1 source in ZIP

CHAPTER 1 / Course Promise: Learn by Building

We will grow one project, not collect disconnected snippets.

Imagine learning to cook: first prepare the kitchen, then learn the tools, then practice techniques, then make a meal. This course grows one project called Erudite the same way.

Each chapter will keep its examples in one chapter component with matching TypeScript, HTML and SCSS files. Important concepts will have three scenarios: simple, realistic, and edge-case/advanced.

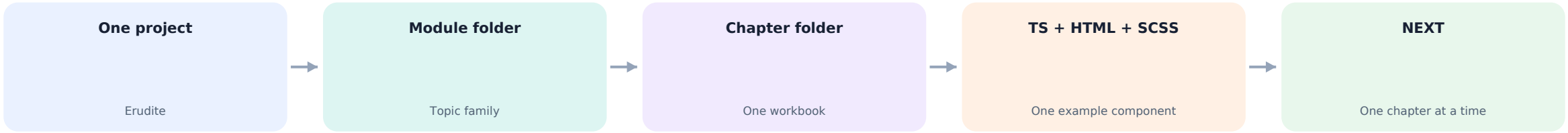
This chapter is the workshop setup. The eventual production app will split shared business features when helpful, but for learning you will see every chapter example together.

CODE / COMMAND / EXAMPLE

```
ERUDITE COURSE RULES
```

ANALOGY: Clear folders work like labelled shelves in a cabinet.

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Your Required Folder Structure

A module is like a shelf, a chapter is a book, and the three source files are its pages.

We will place learning modules under `src/app/modules/`. Each module groups related chapters. Each chapter has its own directory containing the chapter component's `.ts`, `.html`, and `.scss` files.

For example, Chapter 1 lives in `module-01-foundations/chapter-01-setup/`. Chapter 2 will sit beside it and have its own component trio. This makes the learning path predictable.

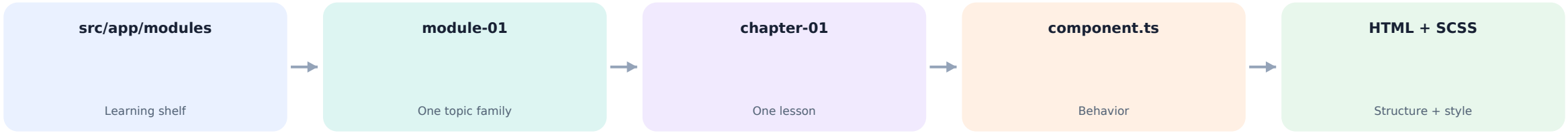
The folder pattern is designed for the course. Later, in a production feature, some reusable controls and services will be split out to avoid duplication.

ANALOGY: Clear folders work like labelled shelves in a cabinet.

CODE / COMMAND / EXAMPLE

STRUCTURE

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / What Chapter 1 Will Build

An empty workshop becomes a working demo screen.

Chapter 1 creates the application and connects one learning component to a route. The screen will contain a welcome panel, an interactive checklist, and a task-status board.

The checklist is a demonstration of UI state. It does not check your actual computer or prove Node.js is installed; the real environment is checked in the terminal.

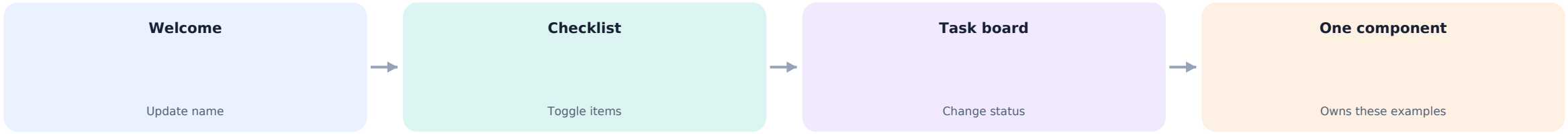
The three scenarios give you a first look at stored values, methods, repeated UI, calculated progress, and status-specific presentation. Later chapters will teach each Angular syntax feature in more depth.

ANALOGY: The root component is the shell; RouterOutlet displays the selected screen.

CODE / COMMAND / EXAMPLE

```
THREE SCENARIOS
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Toolchain Analogy: The Workshop

Angular setup is a group of tools, not one magic installer.

Node.js runs development tools. npm downloads software packages. Angular CLI generates the project and runs builds. VS Code lets you edit files. The browser shows the running web page.

Installing the CLI does not create an app; creating an app does not start the server. Treat each step separately so the failure is easier to locate.

Node.js compatibility depends on the Angular version. Always confirm the current official version matrix instead of blindly copying an old tutorial's Node version.

ANALOGY: The terminal is where you give instructions to the development tools.

CODE / COMMAND / EXAMPLE

```
TOOLS MAP
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Install Node.js and Check Compatibility

The engine must match the machine's required parts.

Install a supported Node.js version for the Angular release you will use. The official Angular compatibility guide lists exact ranges for Node.js, TypeScript and RxJS.

After installing, close and reopen PowerShell or the VS Code terminal so PATH changes are picked up. A command that is missing at this stage is an environment issue, not a component issue.

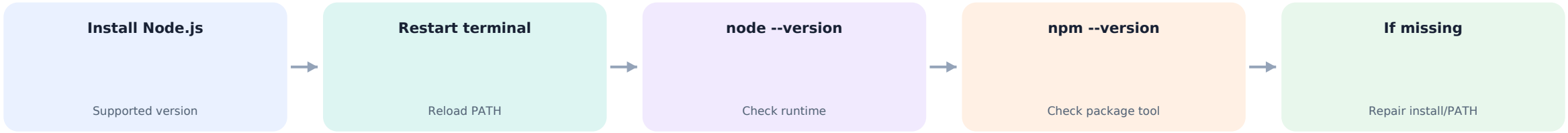
If you already have several Node versions, use an approved version-management approach rather than manually deleting installation folders.

ANALOGY: The terminal is where you give instructions to the development tools.

CODE / COMMAND / EXAMPLE

VERIFY

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Windows Terminal: First Commands

Type only the command; the prompt itself is not part of it.

Open PowerShell or the terminal inside VS Code. Run each command and press Enter. You should see version numbers, not a message that the command is unknown.

Scenario A: both commands work - continue. Scenario B: neither works - install Node.js and reopen terminal. Scenario C: node works but npm fails - repair the Node installation because npm normally comes with it.

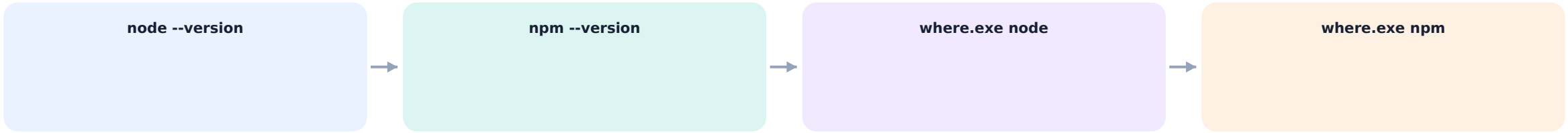
If the version is outside Angular's accepted range, do not force ahead. Check the compatibility matrix and use a supported version.

CODE / COMMAND / EXAMPLE

COMMANDS

ANALOGY: The terminal is where you give instructions to the development tools.

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / The Global ng Command Is Optional for Running

The project uses its local CLI through npm scripts.

If ng is not recognized in the terminal, your global Angular CLI may not be installed or available on PATH. That does not necessarily prevent this project from running, because its package.json has a local CLI dependency.

Use `npm start` and `npm run build` for the normal development workflow. If you later want commands such as `ng generate` directly in any folder, installing the global CLI can be convenient.

The priority is to get the provided workspace running. Do not create a second app just because the global ng command is missing.

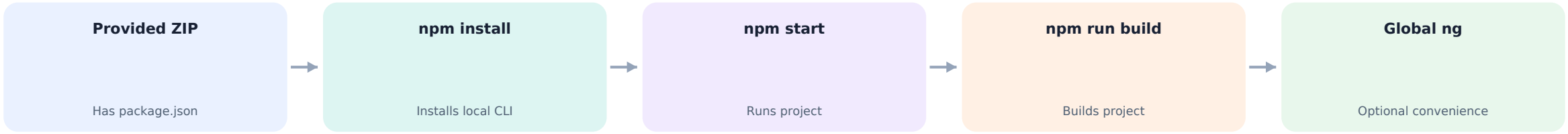
KEY RULE: The supplied workspace is self-contained at source level; npm install supplies its local tools.

COMMANDS

```
# Project-local commands
npm install
npm start
npm run build

# Optional global CLI, later:
npm install -g @angular/cli
ng version
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / If PowerShell Blocks a Script

A door guard may stop a command before that command runs.

Some Windows PowerShell configurations block npm scripts. Read the full error message and use Angular's official installation guidance for your specific case.

Changing execution policy affects which scripts can run, so do not execute a policy-changing command blindly. On a managed work computer, use the approved shell or ask IT rather than bypassing a policy.

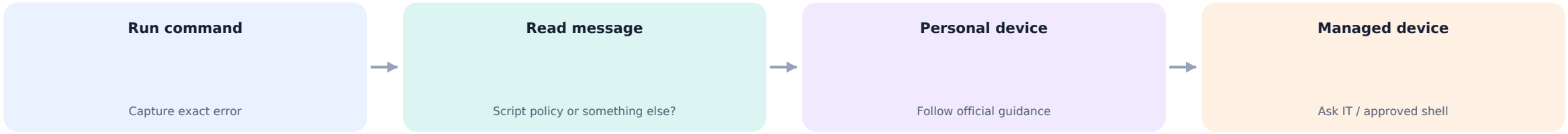
After fixing the terminal issue, rerun `ng version`. Do not start editing Angular code until the toolchain runs consistently.

ANALOGY: Verify the real environment in the terminal; a demo panel is not a system check.

CODE / COMMAND / EXAMPLE

```
DIAGNOSE
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Use the Project You Downloaded

This delivery already contains one Angular workspace and Chapter 1.

Important: the ZIP you received is the project. You do not need to create another one with ng new. Extract it once, then open the extracted Erudite_Angular_Chapter_01_Project folder in VS Code.

The folder you open must directly contain package.json, angular.json, and src/. Opening only `src` is like opening a drawer instead of the whole filing cabinet.

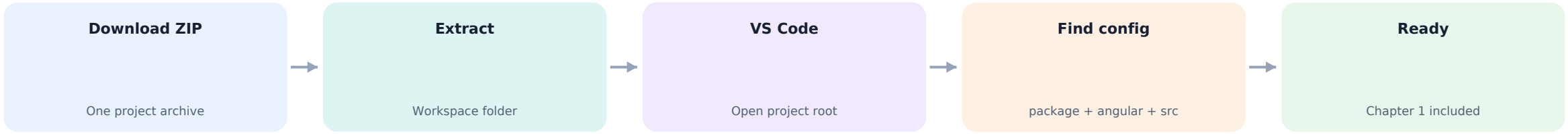
Chapter 1 is already wired into the route table, and its complete TypeScript, HTML and SCSS files are already inside the project.

COMMANDS

```
# Steps in Windows Explorer
1. Download the project ZIP
2. Right-click -> Extract All
3. Open VS Code
4. File -> Open Folder
5. Select Erudite_Angular_Chapter_01_Project
6. Open Terminal -> New Terminal
```

KEY RULE: This archive is the workspace. Do not run ng new and do not create a second project.

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Check the Required Runtime

Angular 22 needs a supported Node.js version.

Before installing packages, check Node.js and npm in the VS Code terminal. The latest Angular documentation lists the compatible runtime range; use a supported Node release rather than guessing.

A missing command usually means Node is not installed correctly or the terminal has not refreshed its PATH. Close and reopen VS Code after installing Node.

You only need the globally installed Angular CLI if you want the ng command outside npm scripts. The downloaded project has its own local CLI dependency, so `npm start` is enough to run it.

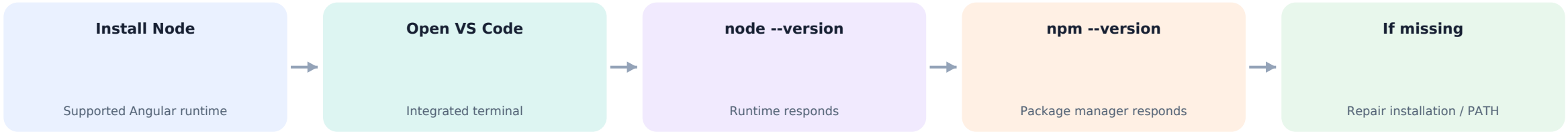
KEY RULE: Verify the terminal first; do not debug Angular templates until Node and npm work.

COMMANDS

```
node --version
npm --version

# Continue only after both commands work
# Compatibility guide:
# https://angular.dev/reference/versions
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Install Dependencies Once

npm install downloads the packages declared by this project.

The project ZIP intentionally does not contain `node_modules`. That directory contains thousands of platform-dependent package files; npm creates it for your computer from `package.json`.

Run `npm install` from the project root - the directory that contains `package.json`. If you run it one level too high or inside `src`, npm may say it cannot find package.json.

This is normally a one-time setup. Later chapters add source folders and sometimes update dependencies; when README instructions say so, run npm install again.

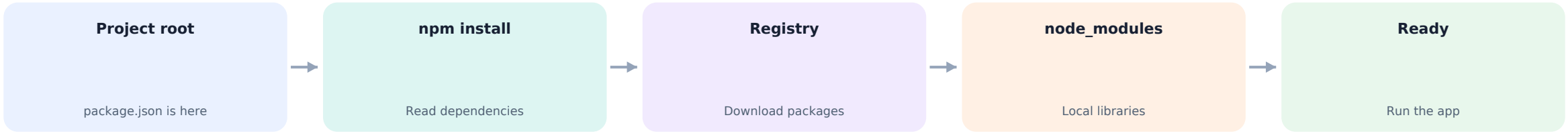
KEY RULE: Run npm install only inside the downloaded project root.

COMMANDS

```
# Run in VS Code terminal at project root
npm install

# The command reads package.json
# and creates node_modules/ locally.
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Start the Existing Project

Use the npm script so the project-local Angular CLI is selected.

Run `npm start` in the VS Code terminal. The local Angular CLI starts the development server. Keep this terminal open while you work; it shows compile errors and tells you which local URL to open.

Browse to `http://localhost:4200/learning/chapter-01`. The root URL redirects to the Chapter 1 learning component. The page should show the welcome panel, checklist, and task status board.

To stop the server, focus the same terminal and press `Ctrl+C`. If port 4200 is busy, start it on another port using the alternative command in the code panel.

KEY RULE: Use npm start for this project; you do not need to run ng new or create another workspace.

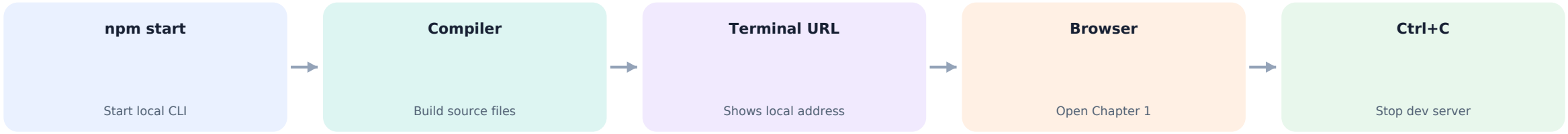
COMMANDS

```
npm start

# Open in your browser:
# http://localhost:4200/learning/chapter-01

# If port 4200 is busy:
npm start -- --port 4300
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / If the First Run Fails

Trace the issue from the earliest broken step.

If `npm install` fails, check Node compatibility and the actual network/proxy error. If the command says package.json is missing, you are not in the project root folder.

If `npm start` fails, read the first useful error and its file path. A template error is different from a missing dependency or port conflict. Fix one issue at a time.

If another app uses port 4200, use the port shown by the terminal or pass port 4300. Do not delete configuration files or node_modules as the first reaction.

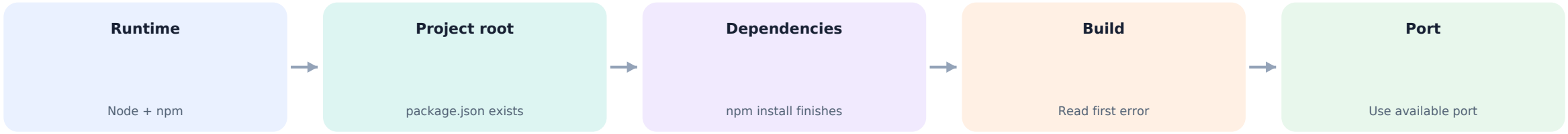
COMMANDS

```
# Run from project root
node --version
npm --version
npm install
npm start

# Alternative port
npm start -- --port 4300
```

KEY RULE: Capture the first real error before changing unrelated files.

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Read the Generated File Tree

You do not need to memorize every file on day one.

Start by locating the root component, route file, app config, and the `main.ts` entry file. The exact root filenames may differ by Angular CLI generation; newer projects may use `app.ts`/`app.html`, while older tutorials use `app.component.ts`/`app.component.html`.

The route file typically lives at `src/app/app.routes.ts`. The root template is where RouterOutlet belongs. The app configuration often connects the route array with `provideRouter(routes)`.

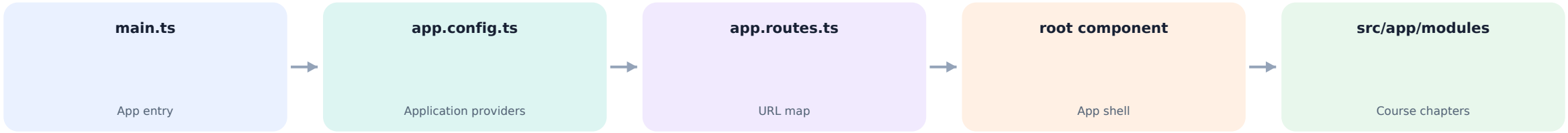
When filenames differ, trust the actual files generated in your workspace and adjust imports consistently. Do not create duplicate app files just because an old tutorial uses another name.

ANALOGY: Clear folders work like labelled shelves in a cabinet.

CODE / COMMAND / EXAMPLE

```
LOOK HERE FIRST
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / How Angular Starts

Opening Angular resembles opening a theatre: start the building, prepare the stage, show the chosen scene.

The browser loads the page shell. `main.ts` starts Angular. The root component becomes the top of the UI tree. The current route chooses which chapter component appears inside RouterOutlet.

`app.config.ts` holds application-wide providers. In a standalone app, `provideRouter(routes)` lets the router use your route array.

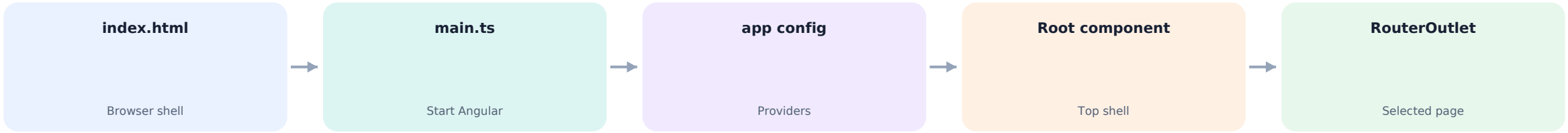
If no UI appears, check the startup chain in order: main entry, app config, root component, route table, and then the chapter component.

ANALOGY: The root component is the shell; RouterOutlet displays the selected screen.

CODE / COMMAND / EXAMPLE

```
BOOT FLOW
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / main.ts: The Starter Switch

Bootstrap means start the app with its root component and settings.

A standalone Angular app commonly calls `bootstrapApplication(App, appConfig)` from `main.ts`. You generally do not change this file in every chapter; it should stay the stable entry point.

If startup fails before routes are considered, inspect the terminal and browser console. A route typo is different from a bootstrap failure.

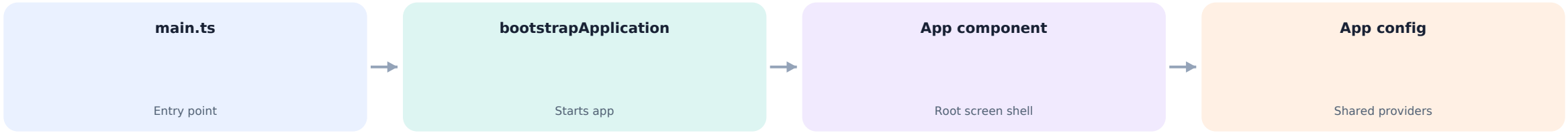
Analogy: main.ts is the key that turns the engine on; app config tells it which shared tools it can use.

ANALOGY: Keep behavior, markup, and styling in their assigned files.

CODE / COMMAND / EXAMPLE

MENTAL MODEL

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / app.config.ts: Shared Setup

An application configuration file is a shared settings board.

The standalone app config registers providers available to the app. For Chapter 1, the key one is `provideRouter(routes)`, which gives Angular the route list.

Later chapters will add HTTP and other services. Do not copy providers from random examples without understanding which feature needs them.

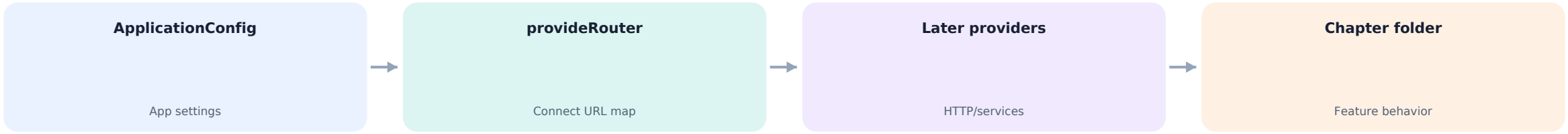
Keep the chapter's behavior in its chapter component; app.config.ts should remain focused on application-wide wiring.

ANALOGY: Keep behavior, markup, and styling in their assigned files.

CODE / COMMAND / EXAMPLE

CONFIG FLOW

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Routes: URL to Screen

A route is a directory sign mapping an address to a room.

A route object connects a URL path to a component or a lazy route definition. Our route will map `/learning/chapter-01` to `Chapter01SetupComponent`.

The root template needs ``, and the root component must import `RouterOutlet`. The router must also be registered using `provideRouter(routes)`.

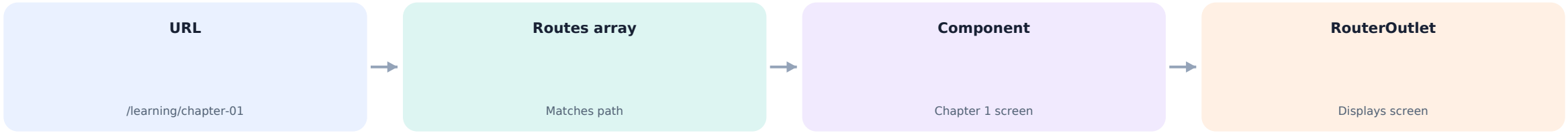
Try typing the chapter URL directly in the browser. Direct navigation confirms that the route is registered rather than reachable only through a button.

CODE / COMMAND / EXAMPLE

ROUTING FLOW

ANALOGY: A route maps a URL to the screen that should appear.

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Root Component: Keep the Lobby Small

The root is the lobby; chapters are separate rooms.

The root class may be named `App` or `AppComponent`. Its role is to provide the shell and outlet for the selected route. The name matters less than correct imports and references.

Replace the generated welcome markup in the root template with ``. Ensure the root standalone component imports `RouterOutlet` from `@angular/router`.

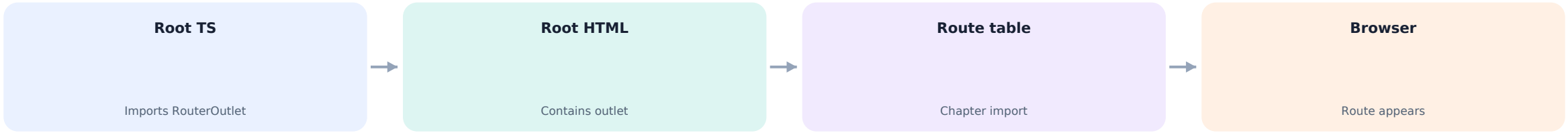
Do not paste the whole chapter template into the root. The root should remain stable while each chapter receives its own component files.

CODE / COMMAND / EXAMPLE

```
ROOT CHECK
```

ANALOGY: The root component is the shell; RouterOutlet displays the selected screen.

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Standalone Component: A Self-contained Room

Standalone means a component can be imported directly rather than declared in a feature NgModule.

The component decorator gives Angular instructions: selector, imports, template path, stylesheet path, and other metadata. The class stores values and methods.

A selector such as `app-chapter-01-setup` is the tag name you could use in another template. A routed page usually gets displayed by the Router rather than inserted manually.

This course uses standalone components. When a component needs Angular directives or other components, those dependencies go in its `imports` array.

ANALOGY: Keep behavior, markup, and styling in their assigned files.

CODE / COMMAND / EXAMPLE

COMPONENT ANATOMY

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Chapter 1 Files Are Already Included

No second component generation is needed for Chapter 1.

The route and component files are already in this workspace. This chapter folder contains the main TypeScript class, HTML template, and SCSS stylesheet. Open those three files in VS Code to study the examples side by side.

The TypeScript file contains the three scenarios: welcome panel, checklist and workflow status. The HTML displays values and sends user events to methods. The SCSS styles cards, buttons, progress and responsive layout.

For Chapter 2, I will add a new sibling folder and send that complete chapter folder after you reply NEXT. We will not create a second Angular project.

COMMANDS

```
src/app/modules/
  module-01-foundations/
    chapter-01-setup/
      chapter-01-setup.component.ts
      chapter-01-setup.component.html
      chapter-01-setup.component.scss
```

KEY RULE: Do not move or recreate this folder; it is already registered in the single workspace.

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / TypeScript: Values and Actions

Properties are stored facts; methods are actions.

`learnerName` is a property. `applyLearnerName()` is a method. The template can show a property and call a method in response to a button event.

Methods should have one clear job. One method updates the name, another toggles a checklist entry, and another changes the task status. This makes the code easier to explain.

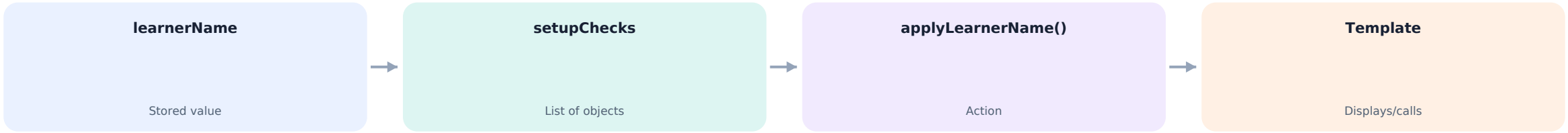
Scenario 1: display a name. Scenario 2: update a selected checklist object. Scenario 3: change the workflow state and its action message.

ANALOGY: Keep behavior, markup, and styling in their assigned files.

CODE / COMMAND / EXAMPLE

SAMPLE

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / HTML: Structure and Meaning

HTML is the room plan; elements define where content and controls go.

Use headings for section titles, paragraphs for explanations, buttons for actions, labels for inputs, and lists for repeated items. Semantic HTML helps keyboard and screen-reader use.

Angular templates add binding expressions to normal HTML. Keep the template descriptive rather than burying complex calculations in it.

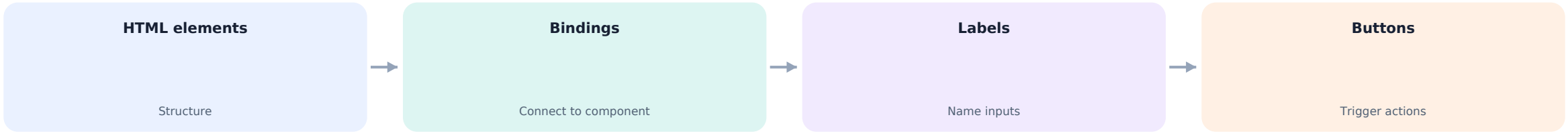
When something is broken, ask: did the HTML trigger the intended method, or is the method correct but the HTML points at the wrong property?

ANALOGY: Keep behavior, markup, and styling in their assigned files.

CODE / COMMAND / EXAMPLE

```
ROLES
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / SCSS: The Component's Appearance

SCSS is the paint plan; it should not decide business results.

SCSS defines spacing, colors, typography, responsive layouts, and status styles. Angular compiles it to CSS and generally scopes component styles to the component view.

Use meaningful class names such as `.status-done` and `.check-row`. A name that describes what something means stays useful even if the color changes.

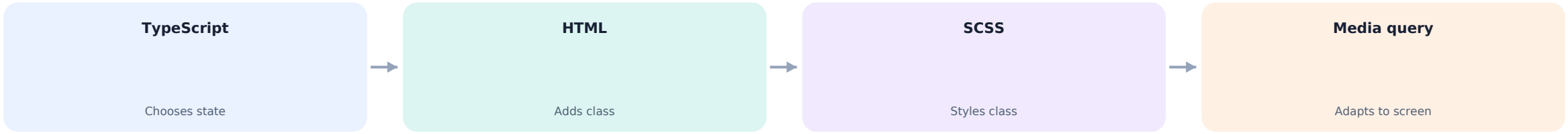
If the text is correct but the badge looks wrong, inspect CSS classes and rules; do not rewrite state logic to fix a styling issue.

ANALOGY: Keep behavior, markup, and styling in their assigned files.

CODE / COMMAND / EXAMPLE

```
VISUAL FLOW
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Interpolation: Display a Value

Interpolation is filling a blank on a printed form with a value.

The syntax `{{ learnerName }}` displays text stored on the component. When the stored value changes, Angular updates the displayed text.

Three scenarios in this chapter: show the learner name; display the completed checklist count; show the current task status label. The same syntax can display different calculated or stored text.

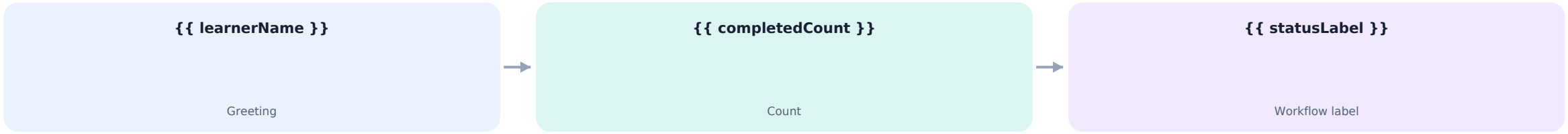
Interpolation is for text display. Property and event bindings do different jobs and will be covered deeply in later chapters.

ANALOGY: Keep behavior, markup, and styling in their assigned files.

CODE / COMMAND / EXAMPLE

THREE EXAMPLES

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Property Binding: Set an Element Value

Property binding sends a value from the component into a rendered element.

`[value]="nameDraft"` keeps the input value connected to a component property. `[style.width.%]="setupPercent"` sets the progress bar width from a calculated number.

Three scenarios: control a progress width; disable a Save button while a request is running; set `aria-pressed` so assistive technology can identify a selected status button.

A property binding describes the current state of an element. It does not run because a user clicked; that is event binding.

ANALOGY: Keep behavior, markup, and styling in their assigned files.

CODE / COMMAND / EXAMPLE

THREE EXAMPLES

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Event Binding: React to User Action

Event binding is a doorbell: the user does something and the component reacts.

`(click)="applyLearnerName()"` calls a method when the button is clicked.
`(input)="onNameDraftChange($event)"` passes the input event into a method.

Three scenarios: apply a name; toggle a checklist item using its id; switch the workflow to Blocked. A named method keeps the HTML readable.

Avoid putting the entire business process inside one template expression. Make the action a method that can be explained, tested, and reused.

ANALOGY: A route maps a URL to the screen that should appear.

CODE / COMMAND / EXAMPLE

THREE EXAMPLES

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Scenario 1: Welcome Panel

A name badge separates draft text from the applied value.

The user types into `nameDraft`. Clicking Update calls `applyLearnerName()`, which trims spaces and updates `learnerName` only if text remains. The heading and avatar initial then refresh.

Tough case A: enter a valid name. Case B: enter only spaces. Case C: repeatedly change the name. These are different outcomes produced by one small method.

The draft/apply separation resembles editing a report title and pressing Apply. It is useful whenever typing should not immediately commit a change.

ANALOGY: A component is one self-contained part of a page.

CODE / COMMAND / EXAMPLE

LOGIC

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Scenario 2: Checklist and Array

An array is a line of cards; each object has the same shape.

`setupChecks` holds objects with an id, title, description, and `complete` flag. `@for` renders one row per object; `track check.id` gives each row stable identity.

`toggleSetupCheck(id)` creates a new array and flips only the chosen item. Because the list is the source of truth, the checkmark and counts can stay synchronized.

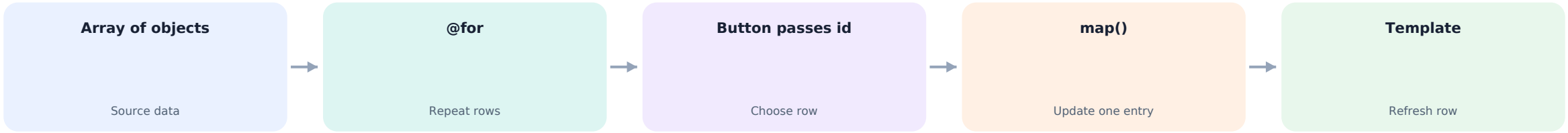
Tough cases: toggle the same entry twice; add a fifth object and ensure progress math still works; remove every item and check that the calculation does not divide by zero.

CODE / COMMAND / EXAMPLE

LOGIC

ANALOGY: Clear folders work like labelled shelves in a cabinet.

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Scenario 2: Derived Progress

Do not store the same fact in three places if you can calculate it from one source.

`completedCount` counts completed rows. `setupPercent` divides that count by the total and rounds to a percentage. Four rows with two complete gives 50%.

Derived state is like calculating a receipt total from line items. If you store the line items, count, and percentage separately, one of them may become outdated.

The getter returns zero when there are no rows. This handles an edge case instead of dividing by zero.

CODE / COMMAND / EXAMPLE

FORMULA

ANALOGY: Keep behavior, markup, and styling in their assigned files.

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Scenario 3: Task Status Board

A task moves through named states like a support ticket.

The `DemoStatus` type allows four known values. `setStatus()` changes the status and updates the latest-action message. The template picks a message and CSS classes based on that state.

Tough cases: switch from Not started to In progress; change it to Blocked; then mark it Done. Only one state should be current at a time.

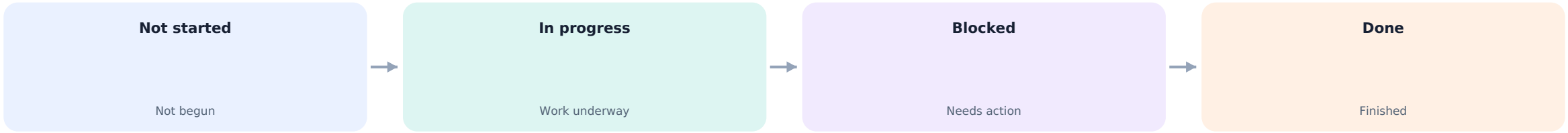
This is only a front-end demonstration. A real workflow must enforce legal transitions and authorization on the backend.

ANALOGY: A route maps a URL to the screen that should appear.

CODE / COMMAND / EXAMPLE

STATE MODEL

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Three Difficulty Levels per Concept

Every concept will use simple, realistic, and edge-case examples.

Level 1 shows the smallest behavior: display one value and update it. Level 2 uses a realistic collection or workflow. Level 3 considers unexpected input, validation, asynchronous behavior, or performance.

This avoids memorizing syntax without understanding what happens when users behave unexpectedly. The course will keep the related examples together in each chapter component.

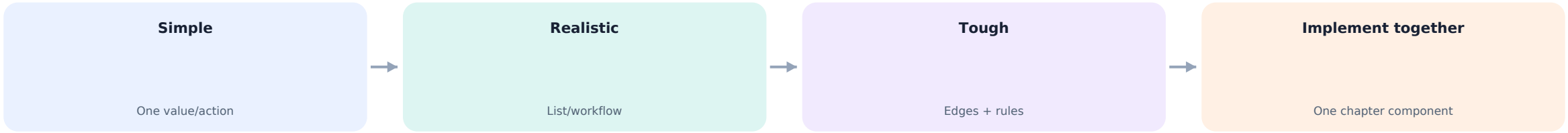
Chapter 1 previews a few binding concepts. Chapter 2 makes components themselves the central subject, with deeper examples of selectors, reuse, inputs, outputs and composition.

CODE / COMMAND / EXAMPLE

```
LEARNING LADDER
```

ANALOGY: A component is one self-contained part of a page.

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / The Chapter Route Is Pre-wired

The URL is already connected to the Chapter 1 component.

Open `src/app/app.routes.ts`. It imports `Chapter01SetupComponent` and maps the path `learning/chapter-01` to that component.

The root component template contains `<router-outlet />`. Think of RouterOutlet as an empty frame in the application shell: the router places the selected chapter inside it.

Do not add a duplicate route for Chapter 1. Later chapter deliveries may include a small route update so a new URL opens the newly added component.

KEY RULE: Chapter 1 routing is already complete; future chapters will add routes to the same file.

COMMANDS

```
// src/app/app.routes.ts
{
  path: 'learning/chapter-01',
  component: Chapter01SetupComponent,
  title: 'Chapter 1 - Project Setup'
}
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Build the Existing Project

A production build is the packed output; source files remain the editable truth.

Use `npm run build` to compile the current application. The output appears under `dist/`, as configured in angular.json.

Never edit files in dist. They are generated output. Edit TypeScript, HTML or SCSS inside src, then rebuild to produce updated output.

If the build reports a template or TypeScript error, read the first relevant error with its path. The development server and production build may show different detail, so a clean build is a useful checkpoint.

COMMANDS

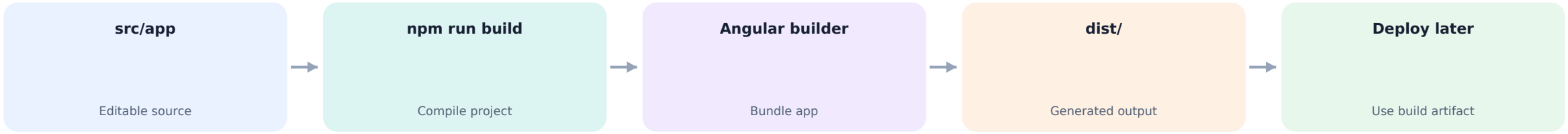
```
# Build this same workspace
npm run build

# Build output is generated under:
# dist/erudite-angular

# Do not edit dist manually.
```

KEY RULE: Change source files under src; treat dist as generated output.

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Git: A Time Machine for Code

Version control lets you return to a known-good project state.

Use `git status` to see changed files and `git diff` to review edits. Commit after the generated app works and again after the Chapter 1 route works.

Do not commit secrets, local credentials, or node_modules. Let the generated `.gitignore` do its job, and inspect any unusual file before staging.

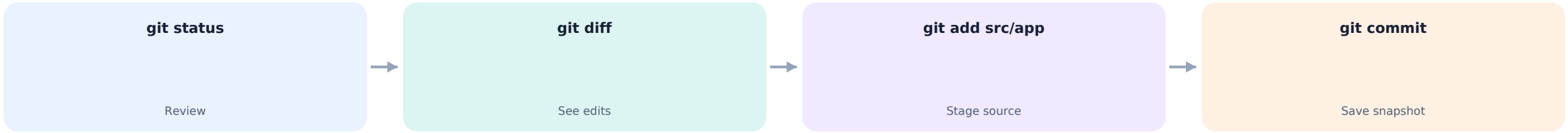
Three scenarios: save the generated starter; save the working chapter; intentionally break a template and use diff to compare the failure with the previous working version.

ANALOGY: Clear folders work like labelled shelves in a cabinet.

CODE / COMMAND / EXAMPLE

SAFE WORKFLOW

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Naming Convention: Fewer Wrong Turns

Consistent labels make files easy to find and imports easier to check.

Use kebab-case for paths and filenames, PascalCase for classes/types, and camelCase for properties/methods. CSS classes should describe meaning rather than color.

Keep `.ts`, `.html`, and `.scss` prefixes consistent. The `templateUrl` and `styleUrl` must match the actual filenames exactly.

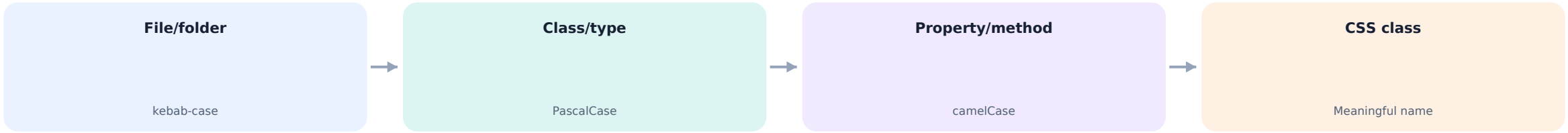
Windows may hide file-case mistakes that appear in other environments. Copy names from the source and avoid multiple variations for one chapter.

ANALOGY: Keep behavior, markup, and styling in their assigned files.

CODE / COMMAND / EXAMPLE

```
NAMING
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / The Global ng Command Is Optional for Running

The project uses its local CLI through npm scripts.

If ng is not recognized in the terminal, your global Angular CLI may not be installed or available on PATH. That does not necessarily prevent this project from running, because its package.json has a local CLI dependency.

Use `npm start` and `npm run build` for the normal development workflow. If you later want commands such as `ng generate` directly in any folder, installing the global CLI can be convenient.

The priority is to get the provided workspace running. Do not create a second app just because the global ng command is missing.

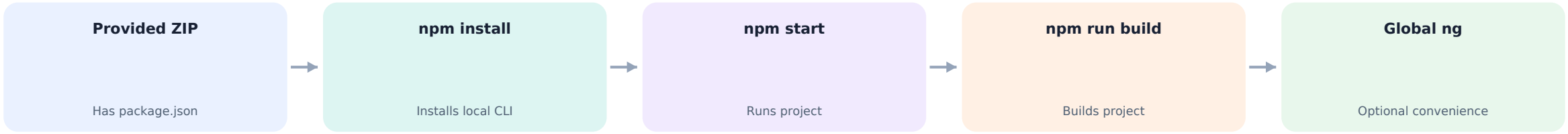
KEY RULE: The supplied workspace is self-contained at source level; npm install supplies its local tools.

COMMANDS

```
# Project-local commands
npm install
npm start
npm run build

# Optional global CLI, later:
npm install -g @angular/cli
ng version
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Error Clinic: Missing Component or Template

An import path is an address; it must point to the actual file.

Check the file exists, the class is exported, the import path is relative to the importing file, and the route actually uses the exported route list.

If Angular cannot locate a template or stylesheet, compare the path in `templateUrl`/`styleUrl` with the real filename. A file can be in the right folder but have the wrong name.

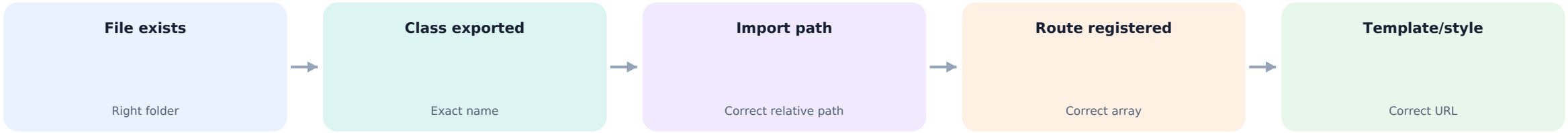
Change one reference at a time and rerun. Avoid creating duplicate files as a workaround for an incorrect import.

ANALOGY: Clear folders work like labelled shelves in a cabinet.

CODE / COMMAND / EXAMPLE

```
CHECK
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Debug by Following the User Action

Debugging is comparing expected behavior with what actually happened.

Write down the action, expected result, and actual result. Example: clicked Mark ready, expected progress to increase, actual progress stayed unchanged.

Trace the path: HTML event -> TypeScript method -> source data -> calculated getter -> displayed binding -> CSS. Stop at the first step that differs from expectation.

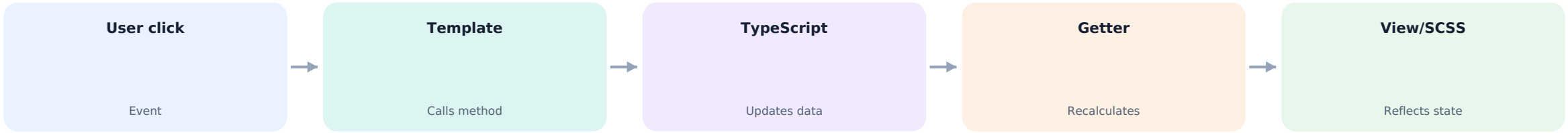
This resembles functional analysis: follow a business process step by step and find where reality diverges from the expected outcome.

ANALOGY: A route maps a URL to the screen that should appear.

CODE / COMMAND / EXAMPLE

TRACE

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Roadmap: How Erudite Will Grow

Each chapter adds one lesson component; the capstone later combines production features.

Module 1 covers setup, components, templates/bindings, control flow, and styling. Module 2 covers directives, pipes, communication and lifecycle. Module 3 covers services, dependency injection and routing.

Later modules include route guards, Reactive Forms, custom controls, HttpClient, interceptors, RxJS, Signals, testing, performance, accessibility, security, and deployment.

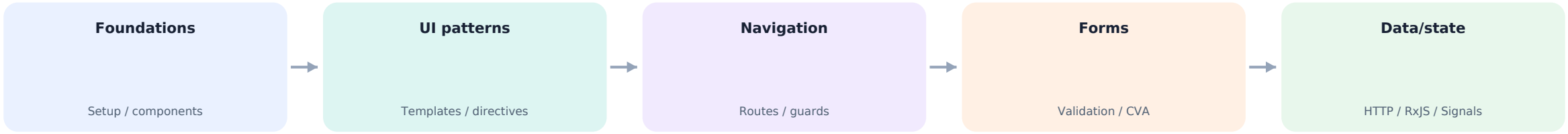
The production capstone will integrate Angular with Spring Boot and build Erudite as a coherent application. Each chapter is a focused laboratory on the way there.

ANALOGY: Clear folders work like labelled shelves in a cabinet.

CODE / COMMAND / EXAMPLE

```
COURSE PATH
```

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Practice: Three Tough Scenarios

Predict what should happen before changing the code.

A) Welcome: reject names longer than 40 characters and display a message. B) Checklist: add a fifth row and confirm the percentage is recalculated. C) Workflow: block the Done action while a required explanation is blank.

Bonus: reset everything; test the component at a phone width; explain why the checklist demo cannot verify actual Node.js installation.

Put each related method in the chapter TS file, its markup in the chapter HTML, and visual states in chapter SCSS. Save a Git checkpoint before attempting the challenge.

CODE / COMMAND / EXAMPLE

```
CHALLENGE
```

ANALOGY: Keep behavior, markup, and styling in their assigned files.

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Completion Checklist

Move on when you can explain the setup, not just copy commands.

You should be able to verify Node/npm, install CLI, create a workspace, run `ng serve`, and recognize the purpose of the main generated files.

You should be able to explain what TypeScript, HTML, SCSS, a component, a route, RouterOutlet, and a workspace each do using plain analogies.

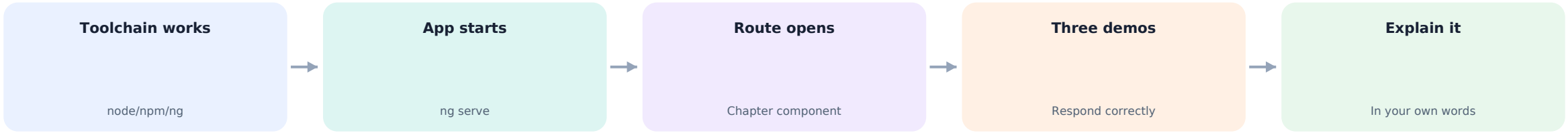
Verify that the Chapter 1 source compiles in your own generated workspace. The starter ZIP is a feature bundle to copy into the workspace; it does not contain node_modules or a preinstalled Angular environment.

CODE / COMMAND / EXAMPLE

```
DONE WHEN
```

ANALOGY: Verify the real environment in the terminal; a demo panel is not a system check.

VISUAL FLOW / MENTAL MODEL



CHAPTER 1 / Official References and NEXT

Use official documentation when CLI prompts or generated filenames differ.

Angular's Installation guide explains local project creation and CLI installation. The Version Compatibility page gives the supported Node.js, TypeScript and RxJS ranges. The CLI overview describes scaffolding, building, testing and maintaining projects.

Chapter 2 is Components in Depth: metadata, selectors, reusability, parent-child composition and component scenarios. We will start with its setup, then build examples from simple to tougher cases.

Reply NEXT when your project starts and the three Chapter 1 scenarios work.

CODE / COMMAND / EXAMPLE

```
OFFICIAL LINKS
```

ANALOGY: Follow the path from source code to component to browser.

VISUAL FLOW / MENTAL MODEL

